

Received February 28, 2022, accepted April 14, 2022, date of publication April 22, 2022, date of current version April 29, 2022.

Digital Object Identifier 10.1109/ACCESS.2022.3169784

Accelerating Falcon on ARMv8

YOUNGBEOM KIM^{ID}, (Student Member, IEEE), JINGYO SONG, (Student Member, IEEE),
AND SEOG CHUNG SEO^{ID}, (Member, IEEE)

Department of Financial Information Security, Kookmin University, Seoul 02707, South Korea

Corresponding author: Seog Chung Seo (scseo@kookmin.ac.kr)

This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) Grant funded by the Korean Government through Ministry of Science and ICT (MSIT) (6G autonomous security internalization-based technology research to ensure security quality at all times) under Grant A2021-0270, and in part by the National Research Foundation of Korea (NRF) Grant funded by the Korean Government through MSIT under Grant 2022R1C1C1013368.

ABSTRACT Falcon is one of the promising digital-signature algorithms in NIST's ongoing Post-Quantum Cryptography (PQC) standardization finalist. Computational efficiency regarding software and hardware is also the main criteria for PQC standardization. In this paper, we present an efficient Falcon software implementation on ARMv8 environment. Until now, most of the software optimization on PQC algorithms have been conducted on 32-bit ARM (Cortex-M4) and typical CPUs (Intel and AMD CPUs). However, ARMv8 including Cortex-A30, 50, and 70 series have been widely used for various IoT (Internet of Things) applications, Edge computing devices, and OBUs (On Board Units) in autonomous driving cars. For optimizing the performance of Falcon, we take full advantage of NEON engine which is a kind of parallel processing unit in ARMv8 MCU. The main computation in Falcon belongs to polynomial multiplications in Complex number domain and Integer domain. Typically, FFT (Fast Fourier Transformation)-based multiplication method and NTT (Number Theoretic Transform)-based multiplication method have been widely used for efficient polynomial multiplications in Complex number domain and Integer domain, respectively. Thus, in order to enhance the overall performance of Falcon, we improve the FFT-based multiplication method and NTT-based multiplication method by utilizing NEON engine in ARMv8. Specifically, we parallelize the overall process (FFT/NTT transformation, pointwise multiplication, and inverse FFT/NTT transformation) of FFT-based polynomial multiplication method and NTT-based polynomial multiplication method with strategically utilizing the NEON engine and vector instructions. Furthermore, we minimize the number of redundant memory accesses during FFT/NTT-based polynomial multiplication by making the most of available registers in NEON engine. Through the proposed parallel FFT/NTT-based polynomial multiplications, the proposed Falcon software provides 15.1% (resp. 18.1%), 16.5% (resp. 17.1%), and 65.4% (resp. 69.4%) of performance improvement in keypair generation, signing, and verification at security level 1 (resp. 5) compared with the reference Falcon implementation submitted to the final round of NIST PQC competition. Furthermore, as far as we know, this is the first optimized implementation of Falcon on ARMv8 environment.

INDEX TERMS Falcon, NIST PQC signature, parallel implementation, memory optimization, ARM/NEON processor, ARMv8, Internet of Things.

I. INTRODUCTION

With the development of a quantum-computing environment, through Shor's algorithm, the problems of prime factorization and discrete logarithms, which are the underlying hardness of public-key cryptosystems, can be solved within polynomial time; therefore, the safety of the existing public-key

cryptosystems, such as RSA and ECC, are recently concerned [1]. Since 2016, the National Institute of Standards and Technology (NIST) has been conducting a standardization project for Post-Quantum Cryptography (PQC) that provides safety even in a quantum-computing environment. In August 2020, Round 3 candidates were selected for Public Key Encryption/Key Encapsulation Mechanism (PKE/KEM) and Digital Signature Algorithm (DSA) schemes, and standardization of final candidates is planned for 2022. Most

The associate editor coordinating the review of this manuscript and approving it for publication was S. M. Rezaul Hasan^{ID}.

of the 3 Round candidates are lattice-based cryptography. Polynomial multiplication, the core operation of lattice-based cryptography, achieves high performance even in embedded platforms because it is accelerated through various algorithms such as Fast-Fourier Transform/Number-Theoretic Transform (FFT/NTT) and Toom-Cook. Among Round 3 DSA schemes, Falcon supports a shorter key length and faster verification speed compared to other DSA schemes; thus, it has advantages that it can be seamlessly integrated into currently used security protocols and applications. For example, recent studies have been conducted to apply PQC-DSA schemes to cryptographic protocols. Since the short signature length and fast verification speed, in WAVE-based V2Verifier, Falcon was selected as the most suitable DSA scheme in the autonomous driving environment [2]. In addition, in TLS 1.3, Falcon is judged that it is the most suitable DSA scheme in a server environment that supports floating-point operations [3].

The Falcon is designed based on the NTRU lattices. Unlike other crypto systems based on the Learning With Error/Learning With Rounding (LWE/LWR), Falcon performs polynomial operations in complex number domain where each coefficient of a polynomial consists of a real and an image part which are represented in floating-point number (Fortunately, since ARMv8 MCU supports generic floating-point arithmetics, we can make use of them to enhance the performance of floating-point arithmetic in Falcon). In addition, in key generation and signature verification process, polynomial operations in integer domain are also required. In summary, the core operation of Falcon is polynomial multiplication in both complex number domain and integer number domain. The reference software of Falcon makes use of FFT/NTT-based multiplication methods for enhancing the performance of polynomial multiplication in each of complex and integer domain, respectively. In particular, key generation process calls various kinds of FFT/NTT to solve NTRU equations, which is a key part of Falcon's performance load. Therefore, in order to accelerate the performance of Falcon, the FFT/NTT need to be optimized. Various implementation studies have been conducted on PQC PKE/KEM and DSA in the ARM architecture, but it is mainly aimed at the ARM-Cortex M series [4]–[6]. As far as we know, the implementation of Falcon in the ARMv8 environment does not yet exist.

ARMv8 architectures are currently being used as a core Micro-Controller Unit (MCU) for mobile, tablet, and autonomous driving. ARMv8 core supports the NEON engine, a parallel processing module, and the Floating-Point (FP) instruction set that can efficiently process FP operations. Thus, in this paper, we present an optimized FFT and NTT based polynomial multiplications, the core operation of Falcon, in ARMv8-A environment. We basically apply data parallelism for processing several coefficients at once by using NEON engine during FFT/NTT-based polynomial operations. For example, 2 (resp. 8) coefficient of a polynomial in complex domain (resp. integer domain) can be

simultaneously processed with NEON engine. Furthermore, regarding FFT multiplication, we optimize FPC functions (FPC-MUL/DIV/ADD/SUB), which are the lowest-level operations of FFT multiplication, by utilizing fused FP instructions and the NEON engine. In addition, the Falcon has different degrees of polynomials for each security level, thus the depth of NTT/FFT at each security level is different. Thus, we apply memory optimization paths taking this into account to minimize the memory access overheads. Through the aforementioned optimization technique, the performance of Falcon has been significantly improved. Our optimized Falcon software library can be utilized for several application where PQC-DSA is required such as V2X (Vehicle to everything) communications requiring authentication of BSM (Basic Security Message), TLS, and DNSSEC.

This work's contributions can be summarized as follows.

- **First Work of Software Optimization about Falcon on ARMv8 Platforms**

Until now, the research on software optimization of Falcon has been only studied on classical CPU (Intel/AMD) and 32-bit ARM Cortex-M4. However, in IoT application, there are many types of devices used. Currently, ARMv8-A based devices like mobile phones, tables, and so on have been widely used for various IoT applications and Edge computing applications. Hence, we concentrate on the software optimization of Falcon, Post Quantum Digital Signature Algorithm (PQDSA), on ARMv8-A based devices. As far as we know, this is the first work on software optimization of Falcon on ARMv8-A based device. At first, we discuss in depth optimization methods of FFT/NTT-based polynomial multiplication, the core operation of Falcon algorithm. As a result of the proposed methods, our Falcon software improved its performance by 15.1% (resp. 18.1%) in KeyGen, 16.5% (17.1%) in Sign, and 65.4% (69.4%) in Verify when compared to Falcon reference software on security level 1 (resp. 5).

- **Optimizing FFT/NTT-based polynomial multiplication on ARMv8-A Devices**

Polynomial multiplication is one of the most time-consuming operation in Lattice-based Post Quantum Cryptosystem. However, Falcon computes polynomial multiplications on not only integer number domain but also complex number while other Lattice-based cryptosystems like Crystals-Dilithium and Crystals-Kyber computing those on only integer number domain. For accelerating the polynomial multiplication in complex number domain and integer number domain, FFT-based polynomial multiplication and NTT-based polynomial multiplication have been widely used, respectively. However, there are room for optimizing them on ARMv8 in order to enhance the overall performance of Falcon. For enhancing the FFT-based and NTT-based polynomial multiplication method, we take full advantage of NEON engine in ARMv8-A MCU. We basically data parallelism approach for processing

Algorithm 1: Key Generation $\text{KeyGen}(\phi, q)$

Require: A monic polynomial $\phi \in \mathbb{Z}[x]$, a modulus q
Ensure : A secret key sk , a public key pk

```

1  $f, g, F, G, \gamma \leftarrow \text{NTRUGen}(\phi, q)$  ▷ Solving the NTRU equation
2  $\mathbf{B} \leftarrow \begin{bmatrix} g & -f \\ G & -F \end{bmatrix}$ 
3  $\hat{\mathbf{B}} \leftarrow \text{FFT}(\mathbf{B})$  ▷ Compute the FFT for each of the 4 components  $\{g, -f, G, -F\}$ 
4  $\mathbf{G} \leftarrow \hat{\mathbf{B}} \times \hat{\mathbf{B}}^*$ 
5  $\mathbf{T} \leftarrow \text{ffLDL}^*(\mathbf{G})$  ▷ Computing the  $\text{LDL}^*$  tree
6 for each leaf  $\text{leaf}$  of  $\mathbf{T}$  do ▷ Normalization step
7    $\text{leaf.value} \leftarrow \sigma / \sqrt{|\text{leaf.value}|}$ 
8  $\text{sk} \leftarrow (\hat{\mathbf{B}}, \mathbf{T})$ 
9  $h \leftarrow gf^{-1} \bmod q$ 
10  $\text{pk} \leftarrow h$ 
11 return  $\text{sk}, \text{pk}$ 

```

2 coefficients (in case of FFT-based polynomial multiplication) and 8 coefficients (in case of NTT-based polynomial multiplication). Furthermore, we make the most of fused multiplication instructions in order to reduce the number of redundant addition and subtraction instructions. The number of memory accesses during polynomial multiplication could be reduced by extending the previous merging technique. Through the proposed several implementation techniques, the performance of FFT-based polynomial multiplication could be improved as 210% (resp. 188%) on security level 1 (resp. 5). The performance of NTT-based polynomial multiplication could be also improved as 250% (resp. 228%) on security level 1 (resp. 5).

• Presenting Optimized Software of Falcon on All Security Parameters

NIST defines the security level of Post Quantum Cryptosystems from 1 to 5 and Falcon provides parameters about security level 1 and 5. Thus, we optimize the performance of Falcon regarding security 1 and 5 parameters. Because the polynomial degrees change as the parameters change, it is necessary to carefully optimize the performance of FFT-based and NTT-based polynomial multiplication. As a result, we have optimized the performance of Falcon by properly optimizing them for parameters and presented the comprehensive performance analysis on all Falcon parameters.

The remainder of this paper is organized as follows Section II introduces the Falcon signature scheme and performance profiling of the reference code, as well as a description of the ARMv8 microcontroller. Section III discusses and analyzes existing implementation research for PQC in an embedded environment. Section IV presents our optimized FFT/NTT-based multiplication implementation for the ARMv8 platform. Section V compares the performance of FFT/NTT-based multiplications and the overall performance

of Falcon in ARMv8 platforms to the reference implementation. Finally, in section VI, we conclude this paper.

II. PRELIMINARIES

A. FALCON

Falcon has a significantly smaller key and signature size than other PQC-DSA competitors. Falcon is designed based on NTRU lattice, so it satisfies compact DSA by using FFT/NTT, which is the core operation of Falcon. The whole process of Falcon is quite complex, but we do our best to explain Falcon DSA. In this article, we express Falcon's whole algorithm based on PQC-specification.

KeyGen process of Falcon consists of two distinct parts. Algorithm 1 shows the overview of the **KeyGen** process. First, it generates F and G that satisfy the NTRU equation through the **NTRUGen** algorithm. f and g are chosen in Gaussian-distribution and check if f^{-1} exists. Then compute $h = gf^{-1} \bmod (q, x^n + 1)$. Finally, **NTRUGen** generates F and G that satisfy $fG - gF = q \bmod \phi$. This process moves to the upper field after performing calculations in the lower field using the field norm map. **NTRUGen** is efficiently computed using FFT/NTT. In addition, since additional operations on one polynomial are used in another, one FFT/NTT representation can be used repeatedly. Therefore, compared to Crystals-Dilithium, which is the same lattice-based PQC-DSA, FFT/NTT are called quite a lot during the **KeyGen** process. Therefore, Falcon's **KeyGen** causes a lot of performance load compared to other PQC-DSAs. The remaining process of **KeyGen** is generating a Falcon Tree for trapdoor sampler operation. In this process, **KeyGen** process performs an LDL^* decomposition on the $\hat{\mathbf{B}} \times \hat{\mathbf{B}}^*$ matrix. Finally, it returns the private key $\text{sk} = (\hat{\mathbf{B}}, \mathbf{T})$ and the public key $\text{sk} = h = gf^{-1} \bmod (q, x^n + 1)$.

In **Sign** process, as shown in Algorithm 2, the signer computes a hash value $c \in \mathbb{Z}_q[x] / \langle x^n + 1 \rangle$. for message m and random seed r . It calculates the short values s_1 and s_2 satisfying $s_1 + s_2 h \equiv c \bmod q$ using the private keys f, g ,

Algorithm 2: Signing $\text{Sign}(m, sk, \lfloor \beta^2 \rfloor)$

Require: A message m , a secret key sk , a bound $\lfloor \beta^2 \rfloor$
Ensure : A signature sig of m

```

1  $r \leftarrow \{0, 1\}^{320}$  uniformly
2  $c \leftarrow \text{HashToPoint}(r || m)$ 
3  $t \leftarrow (-\frac{1}{q} \text{FFT}(c) \odot \text{FFT}(F), \frac{1}{q} \text{FFT}(c) \odot \text{FFT}(f))$   $\triangleright t = (\text{FFT}(c), \text{FFT}(0)) \cdot \hat{B}^{-1}$ 
4 do
5   do
6      $z \leftarrow \text{ffSampling}_n(t, T)$ 
7      $s = (t - z) \hat{B}$ 
8     while  $\|s\|^2 > \beta^2$ 
9        $(s_1, s_2) \leftarrow \text{invFFT}(s)$ 
10     $s \leftarrow \text{Compress}(s_2, 8 \cdot \text{sbytelen} - 328)$ 
11 while  $(s = \perp)$ 
12 return  $\text{sig} = (r, s)$ 

```

Algorithm 3: Verification $\text{Verify}(m, \text{sig}, pk, \lfloor \beta^2 \rfloor)$

Require: A message m , a signature $\text{sig} = (r, s)$, a public key, $pk = h \in \mathbb{Z}_q[x]/(\phi)$, a bound $\lfloor \beta^2 \rfloor$
Ensure : Accept or Reject

```

1  $c \leftarrow \text{HashToPoint}(r || m, q, n)$ 
2  $s_2 \leftarrow \text{Decompress}(s, 8 \cdot \text{sbytelen} - 328)$ 
3 if  $(s_2 = \perp)$  then
4   return reject  $\triangleright$  Reject invalid encodings
5  $s_1 \leftarrow c - s_2 h \bmod q$   $\triangleright s_1$  should be normalized between  $\lceil -\frac{q}{2} \rceil$  and  $\lceil \frac{q}{2} \rceil$ 
6 if  $\|(s_1, s_2)\|^2 \leq \lfloor \beta^2 \rfloor$  then
7   return accept
8 else
9   return reject  $\triangleright$  Reject signatures that are too long

```

F, G . ffSampling process is a randomized rounding process that generates s_1 and s_2 . The main idea of ffSampling process is to calculate the Closest Vector Problem (CVP) in a lower dimension and lift it to a higher dimension. In ffSampling process, vector representation in low dimension is similar to bit-reversed representation of FFT; therefore, it can be easily calculated using the Butterfly Method based on Cooley-Tukey and Gentleman-Sande algorithm [7]–[9]. This mechanism is also applied in LDL^* operations called from ffSampling process; thus, there are different versions of FFT in Falcon, including splitfft . Finally, Sign process outputs the random seed r and the compressed value s of s_2 as a signature.

Verify process of Falcon, as shown in Algorithm 3, has a quite simple structure compared to other PQC-DSAs such as Crystals-Dilithium and Rainbow. In Verify process, s_1 and s_2 are recalculated and checked if $\|(s_1, s_2)\|^2 \leq \lfloor \beta^2 \rfloor$ is satisfied. If that bound $\lfloor \beta^2 \rfloor$ holds for the s_1 and s_2 , then the signature (r, s) is accepted.

B. FFT/NTT-BASED POLYNOMIAL MULTIPLICATION

Fast Fourier Transform (FFT) is an algorithm proposed to fast compute discrete convolutions. FFT-based multiplication

is known as the fastest polynomial multiplication algorithm which processes polynomial multiplication with only $O(n \cdot \log n)$ complexity via the Divide and conquer algorithm. FFT divides the polynomial into its smallest unit by n -th of unity and uses the Divide and conquer algorithm to perform point-by-point multiplication at complexity $O(n)$. Since FFT-based multiplication in Falcon performs multiplication on the elements of a polynomial ring of rational coefficients, unlike NTT, Montgomery-reduction is not required.

Number Theoretic Transform (NTT) is a variant of FFT and is the polynomial multiplication algorithm used by many lattice-based PKE/KEM and DSA schemes that have entered the NIST PQC-competition Round 3 [10]. In general polynomial rings using $\mathbb{Z}_q[x]/\langle 2^n + 1 \rangle$ the condition for using NTT is that n must be a power of 2 and q must be congruent with 1 modulo $2n$. Based on the NTT, the formula for polynomial multiplication is:

$f(x) \times g(x) = \text{NTT}^{-1}(\text{NTT}(f) \odot \text{NTT}(g))$, where $\odot$ is the point-wise multiplication of the coefficients.

The $\log_n$ depth of FFT/NTT of Falcon depends on the polynomial ring used. In the case of Falcon, it has a polynomial of order 512/1024 according to the security level, and since

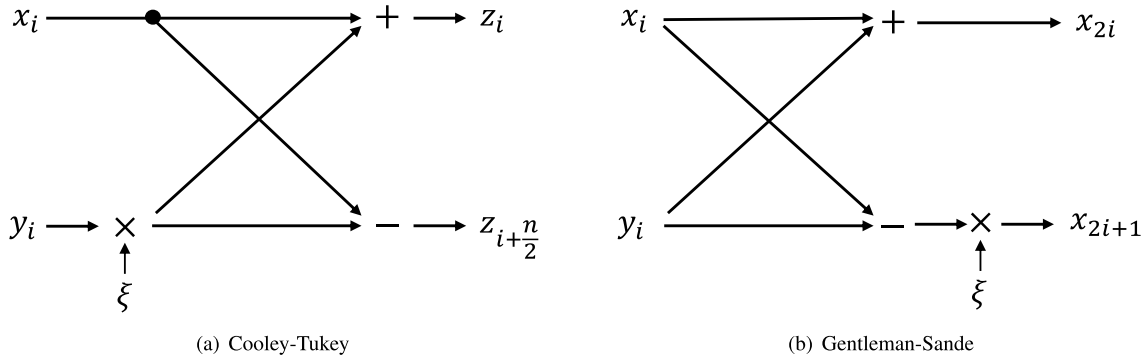


FIGURE 1. Butterfly diagrams for the cooley-tukey and gentleman-sande for FFT/NTT algorithms [7], [8].

FFT/NTT for low dimension is calculated in NTRU equation and `ffSampling`, etc., a considerable number of FFT/NTT are actually called in Falcon. In the case of Falcon, depending on the security level, each polynomial has a depth of 9 to 10 and can be divided until a single rational/integer coefficient. Several methods have been proposed to perform the FFT/NTT. Falcon uses a butterfly technique based on Cooley-Tukey [7] and Gentleman-Sande [8] because it uses a bit-reversal representation at low dimension. Butterfly method is used to match the two transformation methods, and since both algorithms are bit-reverse-based, there is no need to invert additional bits.

Figure 1 shows a diagram of the Cooley-Tukey and Gentleman-Sande algorithms. Since both algorithms are based on bit-reversal, when multiplying n -th of unity, the data tree containing the bit-reversal must be multiplied together ($\xi = n$ -th of unity $\times$ bit-reversal tree). In real implementations, this can be pre-computed and processed to the FFT/NTT algorithm in the form of a lookup table. For the FFT algorithm, the coefficients of the polynomial are rational domains; therefore, no additional operations are required. In this case, the only thing to consider in the FFT algorithm is to determine whether each polynomial belongs to the real or image part of the complex vector. When Cooley-Tukey and Gentleman-Sande are applied, it is divided into real and image parts. However, it should be noted that all parts are used during calculation. In the case of the NTT algorithm, since the main operation is performed in the montgomery domain, additional operations are required. Therefore, inevitably, in Falcon, NTT is slower than FFT. For the Gentleman-Sande algorithm, the coefficients produced when one butterfly operation is completed are doubled. Therefore, in the last layer, all coefficients must be divided by the size of the vector. In the reference implementation, this was processed through shift operation in the montgomery domain.

The number of butterflies of FFT/NTT is calculated in half the order per layer (ie, 256/512 times at 512/1024 security level). Since it is impossible to hold in the register of ARMv8, each polynomial is updated when the butterfly operation on the original address is finished. This causes a lot of memory access load, so it is necessary that an implementation

strategy that can minimize memory access for solving this mechanism. Therefore, in Section IV, we propose the optimal implementation for FFT/NTT implementation in the ARMv8 environment.

C. PERFORMANCE PROFILING OF FALCON REFERENCE CODES

In this section, we profile the Falcon code submitted to the NIST PQC Competition 3 Round; and, we discuss the optimization strategy in this paper. There are three types of reference codes for Falcon. Of these, implementation using AVX instructions is not of our interest because its target platform is the CPU environment. Falcon's reference implementation has an optimized implementation and a generic implementation. The optimization implementation uses a floating-point data type, and the general implementation uses a 64-bit data type for approximating the floating-point operation. Currently, public libraries such as PQM4, PQCclean, and liboqs all have implementations that use approximation-based 64-bit data types for generality [4]–[6]. However, the ARMv8 platform provides floating-point-based instructions in a compact way. Therefore, the target implementation of our optimization technique is an optimized implementation using Falcon's floating-point data type. As far as we know, Falcon's development team did not submit the code for the ARMv8-A series; thus, the code submitted to the 3 Round is the best choice as a result. For profiling, the reference code is compiled on Jetson Xavier's ARMv8.2.

Table 2 shows the profiling performance of Falcon's reference code in Jetson Xavier. Falcon consists of three parts: `KeyGen`, `Sign`, and `Verify`. As shown in Section 1, `KeyGen` generates the NTRU lattices and calls FFT/NTT hundreds of times to solve the NTRU equation. The number of times is variable; however, it is extremely rare to repeat `KeyGen`'s main loop more than 5 times. Various versions of FFT are called in internal functions including `LDL*` that `KeyGen` and `Sign` processes call. For this reason, constructing Falcon's optimization strategy is not easy. In addition, the dividing process of all FFT/NTT in each inner function is variable without working up to the first-order coefficients.

TABLE 1. Summary of FP and ASIMD instruction set in ARMv8 platform [11].

FP Instructions			
Instructions	Operand	Description	Cycles
FMUL	V_n, V_m, V_d	Floating-point Multiplying two vector registers, $V_n = V_m \times V_d$	3
FADD	V_n, V_m, V_d	Floating-point Adding two vector registers, $V_n = V_m + V_d$	1
FSUB	V_n, V_m, V_d	Floating-point Subtracting two vector registers, $V_n = V_m - V_d$	1
FMLS	V_n, V_m, V_d	Floating-point fused Multiply-Subtract from accumulator, $V_n = V_m \times V_d$	3
FMLA	V_n, V_m, V_d	Floating-point fused Multiply-Addition from accumulator, $V_n += V_m \times V_d$	3
NEON ASIMD Instructions			
Instructions	Operand	Description	Cycles
UMULL	V_n, V_{mH}, V_{dH}	Unsigned Multiplying two vector register (Upper Half), $V_n = V_{mH} \times V_{dH}$	3
UMULL2	V_n, V_{mL}, V_{dL}	Unsigned Multiplying two vector register (Lower Half), $V_n = V_{mL} \times V_{dL}$	3
MUL	V_n, V_{mL}, V_{dL}	Multiplying two vector registers, $V_n = V_{mL} \times V_{dL}$	3
ADD	V_n, V_m, V_d	Adding two vector registers, $V_n = V_m + V_d$	1
SUB	V_n, V_m, V_d	Subtracting two vector registers, $V_n = V_m - V_d$	1
UZP1	V_n, V_m, V_d'	Reading even-numbered vector elements from the two source SIMD/FP registers, $[V_m, V_d] \xrightarrow{\text{EvenElement}} V_n$	2
ZIP1, ZIP2	V_n, V_m, V_d'	reading adjacent vector elements from the lower (upper) half of two source SIMD/FP registers as pair, $[V_m, V_d] \xrightarrow{\text{Lower/UpperHalf}} V_n$	2

TABLE 2. Profiling performance of Falcon reference codes on Jetson Xavier (ARMv8.2), FFT(NTT) means FFT(NTT)-based polynomial multiplication.

ALG	All	FFT	NTT	others
KeyGen	13,056,817 (100%)	3,394,772 (26%)	2,350,227 (18%)	7,311,817 (56%)
Sign	580,694 (100%)	278,733 (48%)	- (0%)	301,960 (52%)
Verify	48,023 (100%)	- (0%)	31,214 (65%)	16,808 (35%)

In particular, when the security level is 5 (Falcon-1024), the depth of the layer of FFT/NTT is odd because a polynomial of order 1024 is used. These characteristics impair the generality of optimization strategies and make parallel tasks inconsistent in efficiency.

Our analysis showed that FFT accounted for 26% and 48% of the total clock cycles at KeyGen, Sign, while NTT operations accounted for 18% and 65% in KeyGen and Verify. Since FFT/NTT is based on the same Divide and conquer algorithm, as a result, an optimization method that can efficiently parallelize FFT/NTT-based multiplication is required. For the SHAKE process that generates the seed, there are optimization studies on the keccak algorithm on embedded devices, but none of the studies on the optimal implementation in the ARMv8 platforms as far as we know. It is recommended to use the fully assembled XKCP library generated by the keccak-development team to offload the performance of ExpandA in the ARMv8 environment; however, this paper does not take this into account.

Based on the profiled data, the implementation logic of Falcon's FFT/NTT in Jetson Xavier should be redesigned considering the number of vector registers and floating-point instruction. Operations should be performed while handling the maximum amount of information using the minimum number of registers, and the memory access cycles must be fast at the same time. In addition, since the number of layers of FFT/NTT called from Falcon is constantly changing, The optimization technique taking this into account is essential. Therefore, in this paper, we propose several optimization methods for Falcon in the ARMv8 environment.

D. TARGET DEVICES: ARMv8-A PROCESSOR

ARM architecture is widely used in the embedded industry due to its low power consumption and high performance when compared to previous low-end processors, AVR and MSP. According to their performance, ARM processors are classified into M-series, R-series, and A series levels. Among them, the ARM-A series provides the best performance. Furthermore, the most recent version of an ARM processor is the ARMv8 architecture.

The ARMv8-A series includes an ARM processor as well as a NEON engine. Unlike the NEON engine, the ARM processor does not support parallel processing, but it is adequate for small tasks. Furthermore, the ARM processor includes a barrel shifter, which can hide clock cycles for shift operations in the operand, making it a very powerful technology. The ARM processor's register structure is made up of 64-bit general-purpose registers $x0-x30$, and an A64 instruction set architecture [12] is provided. The NEON engine is a powerful parallel processing engine that supports 128bit vector register $v0-v31$ and ASIMD instructions set architecture [11].

Within a 128-bit vector register, this parallel processing can be done in 64-bit, 32-bit, 16-bit, and 8-bit units. In addition, the target device supports FP instruction to efficiently process floating point operation. Table 1 describes the FP/NEON instructions and clock cycles used in this study to optimize FFT/NTT-based multiplication.

III. RELATED WORKS

Benchmarking research on Post Quantum Cryptography in various environments has continued since the NIST PQC competition started. NIST asked algorithm developers to measure their performance in ARM-Cortex M4 and Artix-7 environments. In addition, not only performance evaluation but also integration with commercial libraries and performance indicators of protocols are one of the important evaluation factors.

PQM4, benchmarking research conducted in an environment on ARM Cortex-M4, presented performance measurement results for PKE/KEM and DSA algorithms. This study provides basic guidance for frameworks targeting embedded platforms [4]. In addition, recently, standalone implementations work has been done such as PQClean project [5]. Additionally, research on applying PQC to the application layer continues [6]. This work can test PQC's algorithm in TLS/SSL protocol using liboqs, a library that integrates PQC and OpenSSL. Also, performance evaluation of PQC-DSA in V2X protocol-related work was recently proposed. V2Verifier tests PQC-DSA in various autonomous driving environments [2]. The key point of related studies is that Falcon is competitive enough because it has a short key and signature length. This advantage means that there is no consideration of packet size in protocols, unlike other DSA algorithms. Since the maximum signature size of Falcon is less than 2000 bytes, it can be sufficiently applied to the WAVE communication standard required in the autonomous driving environment.

Since Falcon's security is based on the long-proven NTRU-lattice, the only consideration for the final competition algorithm is the performance of KeyGen and Sign algorithm. As explained in Section II-C, the main operation of KeyGen and Sign algorithm is the FFT/NTT-based multiplication algorithm. An optimization technique for `fftSampling` for ARM Cortex-M4 environment was proposed in [13], but this technique aims to reduce memory consumption by approaching the memory layout modification point of view. Unfortunately, as far as we know, performance optimization studies targeting FFT/NTT of the Falcon algorithm do not yet exist. On the other hand, various polynomial multiplication algorithms used in LWE/LWR-based schemes, including Crystals-Dilithium, have been actively studied. In this paper, we review various implementation studies for NTT and Toom-Cook-based multiplication in the ARMv8 platforms. Research on PQC in the ARMv8 environment has been actively conducted starting with the implementation for Newhope. [14] was the first to use an unsigned 16-bit integer representation for NTT. In addition, parallel logic for

the NEON engine was proposed. The proposed Newhope implementation was maximally compressed using barrel-shifters etc. Finally, on the ARM Cortex-A53, [14] achieved an 8.3x performance improvement compared to the reference implementation.

Recently, an optimization method for Crystals-Kyber has been proposed. NTT for 16-bit coefficients is used similarly to the method proposed in [15], but the technique for Montgomery reduction is redesigned because of the difference of polynomial-ring. Additionally, all of Crystals-Kyber's core functions for symmetric functions have been accelerated through ARMv8 encryption extensions. In the end, they have achieved a performance improvement of up to 8.6x compared to the reference implementation code. Optimization techniques for NTT are constantly being updated. A method to effectively load and store data for NTT multiplication used by LWE/LWR based algorithms including Crystals-Dilithium has been proposed in [16]. In addition, effective methods for integrating depth-specific layers in NTT have been proposed in [17]. These methods are methods that can be used throughout embedded devices.

Except for ARM-based architectures, there are several studies that implement lattice-based cryptography schemes in various environments. Implementation method studies for various lattice-based schemes have also been conducted on GPU, which is one of the representatives of high-end processors [18]–[22]. However, considerations in the FFT/NTT implementation in a GPU environment are issues such as thread placement per block and warp divergence. Due to the characteristics of the GPU, one NTT per block is calculated in parallel; therefore, the optimization strategy is significantly different from the embedded environment. The core of the FFT/NTT implementation in the embedded environment is a strategy to reduce memory access cost and a strategy to parallelize each coefficient. Since the number of registers and their size is constrained in the embedded environment, there are very small coefficients that can be kept in general registers. In addition, in the case of GPU, the factors of block and thread arrangement rather than the advantage of PTX implementation take up a large cost for performance improvement, but in the case of an embedded environment, using assembly and reducing the number of instructions is an important issue. Another difference from the GPU environment is that the embedded environment uses multiple cores and the cost of creating threads is very high.

There are several studies for lattice-based cryptography schemes on AVR and RISC-V devices. Implementation study of NTRU in AVR environment was conducted in [23]. To perform the optimization in the AVR environment with 8-bit registers, [23] reduced the total cost of the entire cryptographic operation by using the latest parameter set of NTRU and the hybrid technique proposed in [24]. Since it is impossible for the implementations of NTT to be combined in parallel logic on an AVR environment, the authors proposed an implementation methodology that can achieve high performance and resistance to timing attacks by minimizing the use of branches

and conditions. A recent study was conducted that NTT could be used through parameter modification for SABER [25]. In addition, by extending this, studies on the efficiency of incomplete NTT in equipment such as RISC-V have been conducted. In the case of Crystals-Kyber/Dilithium, it is assumed that NTT transformation has been performed for the public matrix, but in the case of SABER, NTT must be applied to the public matrix in order to apply the NTT. Therefore, during actual multiplication, the NTT is performed three times, so the reduction of one layer has a great gain. By using this, [26] proved and implemented that incomplete NTT using 6 layers is the fastest in the RISC-V environment. In RISC-V, no parallelization strategy has been applied, as no SIMD-based implementation is yet available as a basic-instructions.

IV. PROPOSED OPTIMIZATION STRATEGIES

In this section, we present various software-based optimization techniques for FFT and NTT-based polynomial multiplication, which are the core operations of Falcon, using NEON engine in ARMv8-A environment. From the perspective of FFT-based polynomial multiplication, since various FFT multiplications based on FPC operations are utilized in the Falcon algorithm, we basically present parallel implementations for FPC-based operations (FPC-MUL, FPC-DIV, FPC-ADD, FPC-SUB), which are core operations of FFT, using NEON engine. From the perspective of NTT-based polynomial multiplication, we propose a CT Butterfly operation of NTT that maximizes the characteristics of parallelization of NEON engine. In addition, since the Falcon digital signature has a different polynomial degree for each security level, the depth of the NTT and FFT for each security level is different, so a memory optimization strategy considering this is required.

A. FFT-BASED MULTIPLICATION

Some complex number operations are used in signature generation and key pair generation. This complex number operation can be approximated by IEEE 754 floating-point operation. Therefore, it is encoded as a 64-bit Double type and the operation is performed. A Falcon digital signature provides FPC-MUL, FPC-DIV, FPC-ADD, and FPC-SUB, which are the lowest level operations to efficiently process these complex number operations. These FPC-MUL, FPC-DIV, FPC-ADD, and FPC-SUB operations are mixed to form many FFT multiplication (Standard FFT, Merging FFT, Split FFT, LDL FFT, etc.) operations. Therefore, these functions should be optimized to maximize the performance of FFT-based multiplication. However, in an environment where some floating point instructions are not supported, more complex operations must be performed to process the floating point operation, but the target device supports the floating point operation as well as NEON engine capable of parallel processing. Thus, we present parallel implementation technique for FPC-MUL, FPC-DIV, FPC-ADD, and FPC-SUB operations, which are core operations of FFT multiplication

by utilizing FP instruction and NEON engine. Thus, we propose parallel implementations that simultaneously processes two coefficient for FPC functions, which are core operations, by utilizing FP instructions and NEON engine.

Unlike integer arithmetic, complex number arithmetic is divided into real part and image part to perform operations. In order words, $(a+bi) + (c+di)$ operation becomes $(a+c) + (b+d)i$ by dividing into real part and image part. Algorithm 4 shows the parallel implementation of the proposed FPC functions. In Algorithm 4, Input data are vector registers storing two complex numbers, and Output data are vector registers storing operation results for two complex numbers. First, the FPC-MUL function is a function that multiplies two complex numbers, and is the lowest-level function that is used as a core in KeyGen and Sign of Falcon. Step 1-2 is the part where the Real Part is calculated, and Step 3-4 is the part where the Image Part is calculated. Through steps 1-4, the result of complex multiplication of two coefficients in the 128-bit vector register is returned. We designed the FPC-MUL function compactly by eliminating arithmetic instructions such as SUB and ADD that may additionally occur by using the computational characteristics of the FPC-MUL function and fused MUL (FMLA, FMLS).

FPC-DIV is a division operation on two complex numbers and has the largest performance overheads among FPC functions. In addition, the FPC-DIV function is frequently used in LDL sampling of signature generation, and needs optimization to improve performance in signature generation. We present a parallel implementation of the FPC-DIV function for two coefficients utilizing the NEON engine. Steps 5-14 show a parallel implementation of the division process on two complex numbers. By utilizing FMLS and FMLA instructions as much as possible, ADD and SUB instructions that may occur similar to FPC-MUL functions are minimized to improve performance. Finally, Step 15-16 and Step 17-18 are parallel implementations of the proposed FPC-ADD and FPC-SUB, and addition and subtraction of two complex numbers can be easily performed in parallel with only FADD and FSUB instructions.

B. NTT-BASED MULTIPLICATION

In general, implementations of cryptographic algorithms can be done sequentially or in parallel. Although sequential execution compute one operation at a time, the parallel implementation uses a parallel module supporting the SIMD instruction set to perform multiple operations at a time. A parallel implementation ideally yields an $X\%$ performance improvement over sequential execution, where X is the number of simultaneous operations. For this reason, parallel implementations typically achieve more accelerated computational processing than serial implementations. Therefore, we present a parallel implementation technique of CT Butterfly that performs 8 coefficients simultaneously considering the size of NEON vector register and modulus of Falcon.

Several NTT algorithms are used in KeyGen, Sign, and Verify of Falcon. Compared to other PQC digital signature

Algorithm 4: Proposed FPC Parallel Implementation Using NEON Engine

Input : $X_{Re}, X_{Im}, Y_{Re}, Y_{Im}$	
Output: $v28_{(Re)}, v29_{(Im)}$	
* Alg: FPC-MUL : $(a + bi) \times (c + di) = (ac - bd) + (ad + bc)i$	
1 FMUL v28.2d, \XRe\().2d, \YRe\().2d	▷ Real Part Operation ac
2 FMLS v28.2d, \XIm\().2d, \YIm\().2d	▷ Result $\rightarrow (ac - bd)$
3 FMUL v29.2d, \XRe\().2d, \YIm\().2d	▷ Image Part Operation ad
4 FMLA v29.2d, \XIm\().2d, \YRe\().2d	▷ Result $\rightarrow (ad + bc)$
* Alg: FPC-DIV : $(a + bi)/(c + di) = ((ac + bd) - (ad - bc)i)/(c^2 + d^2)$	
5 FMUL v28.2d, \YRe\().2d, \YRe\().2d	▷ c^2
6 FMLA v28.2d, \YIm\().2d, \YIm\().2d	▷ $c^2 + d^2$
7 FDIV v28.2d, v31.2d, v28.2d	▷ $1/(c^2 + d^2)$
8 FMUL \YRe\().2d, v28.2d, \YRe\().2d	▷ $c/(c^2 + d^2)$
9 FSUB v29.2d, v30.2d, \YIm\().2d	▷ $-d$
10 FMUL \YIm\().2d, v29.2d, v28.2d	▷ $-d/(c^2 + d^2)$
11 FMUL v28.2d, \XRe\().2d, \YRe\().2d	▷ Real Part Operation $ac/(c^2 + d^2)$
12 FMLS v28.2d, \XIm\().2d, \YIm\().2d	▷ Result $\rightarrow (ac + bd)/(c^2 + d^2)$
13 FMUL v29.2d, \XRe\().2d, \YIm\().2d	▷ Image Part Operation $(-ad)/(c^2 + d^2)i$
14 FMLA v29.2d, \XIm\().2d, \YRe\().2d	▷ Result $\rightarrow (-ad + bc)/(c^2 + d^2)$
* Alg: FPC-ADD : $(a + bi) + (c + di) = (a + c) + (b + d)i$	
15 FADD v28.2d, \XRe\().2d, \YRe\().2d	▷ Real Part Result $\rightarrow (a + c)$
16 FADD v29.2d, \XIm\().2d, \YIm\().2d	▷ Image Part Result $\rightarrow (b + d)$
* Alg: FPC-SUB : $(a + bi) - (c + di) = (a - c) + (b - d)i$	
17 FSUB v28.2d, \XRe\().2d, \YRe\().2d	▷ Real Part Result $\rightarrow (a - c)$
18 FSUB v29.2d, \XIm\().2d, \YIm\().2d	▷ Image Part Result $\rightarrow (b - d)$

algorithms, Falcon has the advantage of significantly faster signature verification speed. Thus, to further emphasize this advantage, we focus on the parallel implementation of the mq-NTT algorithm utilized in the signature verification process of the Falcon. The NTT algorithm used in the rest of KeyGen and Sign can be implemented similarly. Algorithm 5 shows the proposed parallel implementation technique for CT Butterfly operation for the mq-NTT algorithm. Input of Algorithm 5 is the vector registers each storing 8 coefficients, and Out corresponds to the vector registers storing 8 coefficients for which CT-Butterfly operation has been completed. Also, CT-Butterfly is an algorithm for accelerating NTT in parallel, and consists of mq-montymul, mq-add, and mq-sub operations.

First, the code corresponding to Step 1-23 is to process mq-montymul. In Step 1-2, the multiplication between Zetas and the In1 is performed in 16-bit units by separating the 128-bit vector register into 64-bit unit. Since the parallel unit has been changed from 16-bit to 32-bit due to multiplication operation, the subsequent operation processing must be divided into two parts and performed separately. Step 24-25 is the process of converting an 8×16 -bit input vector register into two 4×32 -bit vector registers to perform mq-add, which processes operations in 32-bit units. Steps 27-39 perform the parallel processing of mq-add in a 128-bit vector register by 32-bit units. When the mq-add operation is completed, since the CT-Butterfly of one part is finished, it is necessary to change the parallel unit from 4×32 -bit to 8×16 -bit again,

which is Step 40. Finally, CT-Butterfly operation is finalized through mq-sub. Similar to mq-add, since operations are performed in 32-bit units, the process of integrating the parallelization of the resulting two vector registers is required. CT-Butterfly operation is a key operation in NTT, and the proposed parallel implementation has faster performance by processing multiple operations simultaneously compared to a serial implementation.

C. MEMORY OPTIMIZATION STRATEGY: FFT/NTT

Falcon guarantees security based on NTRU lattice, so the polynomial degree increases with each security level. In other words, Falcon-512 operates on polynomials of the 512 degree, and Falcon-1024 operates on polynomials of the 1024 degree. If the degree of polynomial is different according to the security level, the depth of the FFT/NTT is different. Since memory access is expensive in the embedded environment, a memory optimization strategy considering the depth of such FFT/NTT is required. Thus, we present a memory optimization path for FFT/NTT-512, FFT/NTT-1024. We designed the memory optimization paths by maximizing the merging proposed in previous study [27]. For example, based on FFT-512, it is possible to save 16 LD1 and ST1 memory access instructions each in Real and Image Part for 2 depths compared to Standard implementation. In addition, by maximizing NEON vector registers v0 – v31, coefficients of polynomial are held and processed in all vector registers

Algorithm 5: Proposed Parallel CT Butterfly Method of NTT Multiplication Utilizing NEON Engine

Input : $In0, In1$
Output: $Out0, Out1$
*** Alg: mq-montmul (step 1-23)**
 $//z = x \times y \quad z \in 2^{32}, x, y \in 2^{16}$
1 UMULL v26.4s, \In1\().4h, v28.4h $\triangleright$ v28: Zetas
2 UMULL2 v27.4s, \In1\().8h, v28.8h
 $//w = ((z \times Q0I) \& 0xFFFF) \times Q, w \in 2^{32}$
3 MUL v25.4s, v26.4s, v30.4s $\triangleright$ v30: Q0I
4 MUL v24.4s, v27.4s, v30.4s
5 AND v25.16b, v25.16b, v20.16b $\triangleright$ v20: 0xFFFF
6 AND v24.16b, v24.16b, v20.16b
7 MUL v25.4s, v25.4s, v29.4s $\triangleright$ v29: Q
8 MUL v24.4s, v24.4s, v29.4s
 $//z = (z + w) \gg 16$
9 ADD v26.4s, v26.4s, v25.4s
10 ADD v27.4s, v27.4s, v24.4s
11 USHR v26.4s, v26.4s, #16
12 USHR v27.4s, v27.4s, #16
 $//z- = Q$
13 SUB v26.4s, v26.4s, v29.4s $\triangleright$ v29: Q
14 SUB v27.4s, v27.4s, v29.4s
 $//z+ = Q \& -(z \gg 31)$
15 USHR v23.4s, v26.4s, #31
16 USHR v22.4s, v27.4s, #31
17 MOVi v24.16b, #0
18 SUB v23.4s, v24.4s, v23.4s
19 SUB v22.4s, v24.4s, v22.4s
20 AND v23.16b, v29.16b, v23.16b $\triangleright$ v29: Q
21 AND v22.16b, v29.16b, v22.16b
22 ADD v23.4s, v23.4s, v26.4s
23 ADD v22.4s, v22.4s, v27.4s
*** Alg: Parallel Unit Conversion $2^{32} \rightarrow 2 \cdot 2^{16}$**
24 MOVi v24.16b, #0
25 ZIP1 v25.8h, \In0\().8h, v24.8h
26 ZIP2 v24.8h, \In0\().8h, v24.8h
*** Alg: mq-add (step 27-39)**
 $//d = x + y - Q, d \in 2^{32}, x, y \in 2^{32}$
27 ADD v19.4s, v23.4s, v25.4s
28 ADD v18.4s, v22.4s, v24.4s
29 SUB v19.4s, v19.4s, v29.4s $\triangleright$ v29: Q
30 SUB v18.4s, v18.4s, v29.4s
 $//d+ = Q \& -(d \gg 31), d \in 2^{32}$
31 USHR v21.4s, v19.4s, #31
32 USHR v26.4s, v18.4s, #31
33 MOVi v27.16b, #0
34 SUB v21.4s, v27.4s, v21.4s
35 SUB v26.4s, v27.4s, v26.4s
36 AND v21.16b, v21.16b, v29.16b $\triangleright$ v29: Q
37 AND v26.16b, v26.16b, v29.16b
38 ADD v19.4s, v19.4s, v21.4s
39 ADD v18.4s, v18.4s, v26.4s
*** Alg: Parallel Integration**
40 UZP1 \Out0\().8h, v19.8h, v18.8h
*** Alg: mq-sub (step 41-51)**
 $//d = x - y, d \in 2^{32}$
41 SUB v23.4s, v25.4s, v23.4s
42 SUB v22.4s, v24.4s, v22.4s
 $//d+ = Q \& -(d \gg 31)$
43 USHR v24.4s, v23.4s, #31
44 USHR v25.4s, v22.4s, #31
45 MOVi v26.16b, #0
46 SUB v24.4s, v26.4s, v24.4s
47 SUB v25.4s, v26.4s, v25.4s
48 AND v24.16b, v24.16b, v29.16b $\triangleright$ v29: Q
49 AND v25.16b, v25.16b, v29.16b
50 ADD v24.4s, v24.4s, v23.4s
51 ADD v25.4s, v25.4s, v22.4s
*** Alg: Parallel Integration**
52 UZP1 \Out1\().8h, v24.8h, v25.8h

$v0 - v31$ after some depth, eliminating all possible memory accesses.

Figure 2 shows the proposed memory optimization path of FFT/NTT-512 and FFT/NTT-1024. FFT is divided into each Real and Image Part for the polynomial to be transformed, and each FFT is performed. For example, FFT-512 performs each transformation of a 256 degree of polynomial in the Real Part and a 256 degree of polynomial in the Image Part. Thus, in FFT-512 and FFT-1024, FFT depth for 256 degree of polynomial and 512 degree of polynomial should be considered. In FFT-512, from 256 degree of polynomial transformation to 16 degree of polynomial transformation, it reaches with only two merging. After that, memory access is not required until the FFT is completed by applying the register holding technique. In FFT-1024, the depth is odd, so it is impossible to reach the register holding depth by utilizing merging technique only. Therefore, we added a standard implementation technique to reach register holding depth. Because the NTT is performed in the integer domain, it is not split into two parts for polynomials like the FFT. NTT-512 and NTT-1024 perform transformation of polynomial degree supported by each security level. Since depth is odd in NTT-512, like FFT-1024,

standard implementation is added to suggest optimal memory path. In NTT-1024, it reaches register holding with only 3 merging. Through the proposed memory optimization path, we further accelerated performance of FFT/NTT by minimizing memory access instructions such as LD1 and ST1 instructions.

Figure 3 shows the details of the memory optimization technique applied to Falcon's FFT/NTT. Each point in Figure 2 represents a Falcon's polynomial of the 64th degree, and the brown line is the memory access that occurred in our FFT/NTT implementation. The black lines are the butterflies that are first-order realized in the merging technique. we approach Figure 3 from two perspectives.

The first is a column-wise fashion FFT/NTT implementation. The merging technique can be described as keeping the coefficients of the polynomials in the same register so that the polynomials needed for the butterfly to be computed in the next layer can be computed without additional memory access. Therefore, when this method is used, the butterfly process is performed on the second layer as soon as all the coefficients to be used in the second layer are calculated, unlike the standard implementation. If this is repeatedly

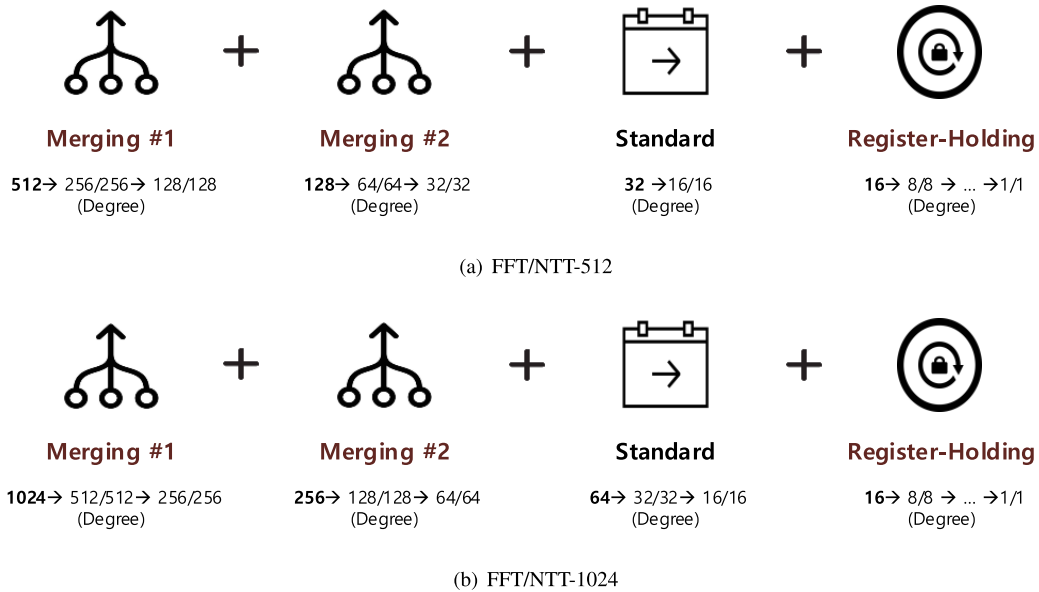


FIGURE 2. Proposed memory optimization path of FFT/NTT for 512/1024 security level.

performed, only two memory accesses occur for the polynomial when operating the actual two layers. In the case of the standard implementation, when operating two layers, three memory accesses for polynomial occur, so the reduction in memory access by one may seem small, but in the case of Falcon, it is a polynomial of order 512, and a total of three (Inv)FFT/(Inv)NTT operation is required for FFT/NTT-based multiplication. Therefore, the actual decreasing clock cycles cannot be neglected. In addition, since the merging technique is used twice (Merging#1/Merging#2), memory access to two layers is omitted from the entire FFT/NTT transform.

The second is the register-holding technique. The above technique is performed in the lower layer of FFT/NTT, and when implementing the divide and conquer algorithm, the memory access caused by interference with the nearest coefficient is offset by maintaining the coefficients in the vector register. Applying this, no memory access occurs, unlike the standard implementation. Similar to merging, but different in that register-holding is a row-wise fashion implementation and has a different number of affected layers. In the lower layer of FFT/NTT, the coefficients to be calculated are attached to each other; therefore, only two memory access costs are incurred.

Since FFT/NTT implementations are essentially a divide and conquer of polynomials, we need to access all coefficients along with the order of each polynomial. Generally, the polynomial approach is done in a row-wise fashion. However, in order to minimize the memory access cost, it should be approached in a column-wise fashion. In our analysis, the most efficient number of layers for merging was two. Since this is not related to the order of polynomials, the number of combined layers is maintained even at the 1024 security level. From a merged depth perspective,

we present Merging#1 and Merging#2. In the case of 1024 security level, it seems that the merging technique can be applied once again (Merging#3), but due to the characteristics of FFT/NTT, it is necessary to move the coefficient of the polynomial in the register for applying to the butterfly method. As we analyzed, introducing Merging#3 incurs an additional computational cost, so it is most efficient to implement a butterfly less than the 64th degree polynomial at 1024 security level through the standard/register-holding techniques.

Even if ARMv8 supports Floating-Point (FP) instructions, there are cases in which NEON engine's FP operation cannot be used for actual compilation reasons such as gcc options. However, the approximation operation, using integer data type, that does not use the FP operation does not convert the number of bits because only the data type difference exists (double $\rightarrow$ uint64_t). Accordingly, the maximum polynomial coefficient that can be maintained in the NEON register is the same as when using the FP operation with two coefficients even when using the approximation operation based on the 64-bit integer data type. Therefore, the number of merged layers of FFT and the Register-Holding technique are still applicable. Therefore, this technique can be applied to any device that has 128-bit parallel operation support register.

D. POINT-WISE MULTIPLICATION

To implement Point-wise multiplication in the ARMv8 environment, we take full advantage of the parallelization technology of the NEON engine to accelerate optimization. For FFT algorithm with FP coefficients in 64-bit units, ARMv8 can keep up to two in the NEON register; so, the two coefficients are processed simultaneously. Conversely, since NTT algorithm is processed by integer coefficients in 16-bit units, each

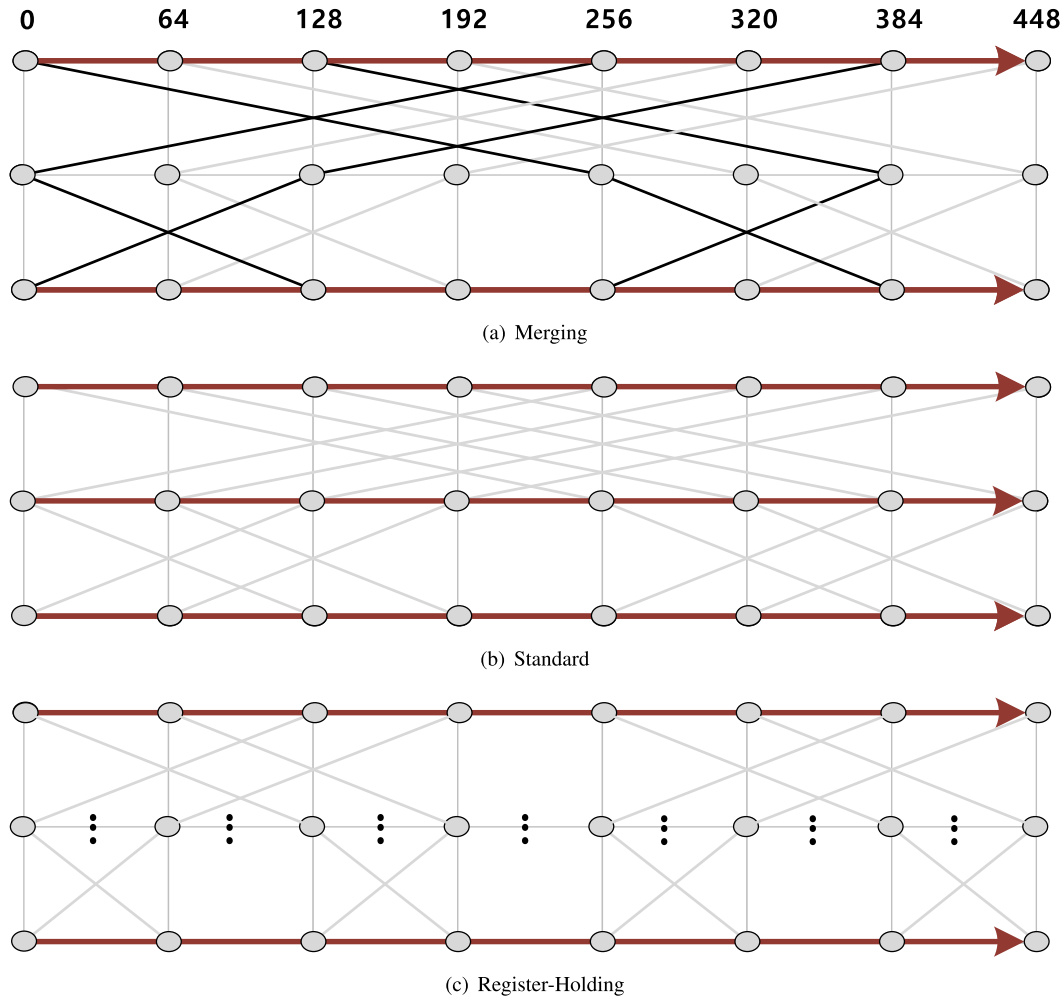


FIGURE 3. Proposed memory optimization technique for FFT/NTT algorithm.

coefficient can be kept in the NEON register, up to 8; therefore, eight coefficients are calculated simultaneously. In the NTT implementation, Montgomery reduction operations are still performed. The coefficients of polynomials which is the result of NTT are still in the Montgomery domain.

V. EVALUATION

A. TARGET DEVICE: JETSON XAVIER

In this section, we evaluate the performance of Falcon software incorporating the proposed FFT/NTT optimization. We measured performance on Jetson Xavier boards. Jetson Xavier board is an on-board unit that is widely used in autonomous driving environments. The Jetson Xavier CPU has 8 ARMv8.2 cores and the same out-of-order pipeline as ARMv8. ARMv8.2 supports half-precision floating-point processing, RAS, statistical profiling, and an improved memory model architecture when compared to ARMv8 [29]. It has a 64KB L1 data cache, a 128KB L1 instruction cache, and a 2MB L2 cache, and it can run at up to 2.26GHz. We used VScode development tools in Ubuntu

20.04 environment, and used AArch64-gnu-gcc 7.5.0 compiler. Until now, most of research of falcon has been conducted in the ARM Cortex-M4 environment, and since there are no research results in the ARMv8 environment, we provide detailed comparison with reference implementation of Falcon submitted to the NIST Round 3 Submission. The reference implementation is composed of C language which uses FP data type and the `-O3` option is applied when compiling, and the proposed implementation is composed of assembly language.

B. PERFORMANCE ANALYSIS FOR NTT/FFT-BASED POLYNOMIAL MULTIPLICATION

As mentioned in Section II, since the ARMv8 platforms supports floating-point operations, the reference code, we are comparing, is the optimized version of Falcon submitted to 3 Round. Table 3 provides detailed comparison of Falcon Round 3 Submission and FFT/NTT-based multiplication of the proposed Falcon software. Compared to reference implementation in the NTT and InvNTT process, we basically

TABLE 3. Performance comparison of the NTT/FFT of Falcon on Jetson AGX Xavier.

Security Level	Implementation (language)	FFT/NTT	Point-Wise multiplication	InvFFT/NTT	FFT/NTT-based multiplication
1	[9]'s FFT (C)	4,296	235	5,186	9,717
	[9]'s NTT (C)	9,305	360	13,265	22,930
	Our FFT (Asm)	1,211	210	1,355	2,776
	Our NTT (Asm)	3,351	331	3,712	7,394
	Speed-Up ([9] vs Our)	+254/177%	+11/9%	+282/257%	+250/210%
5	[9]'s FFT (C)	8,916	442	10,331	19,689
	[9]'s NTT (C)	19,567	695	27,068	47,330
	Our FFT (Asm)	2,781	402	2,810	5,993
	Our NTT (Asm)	7,685	647	8,088	16,420
	Speed-Up ([9] vs Our)	+219/154%	+10/8%	+267/234%	+228/188%

TABLE 4. Performance comparison of falcon on jetson AGX xavier.

Security Level	Implementation (language)	Algorithm	KeyGen	Sign	Verify
1	[9] (C)	Falcon	13,056,817	580,694	48,023
	Our work (Asm)		11,341,553	498,561	29,023
	Speed-Up ([9] vs Our)		+15.1%	+16.5%	+65.4%
2	[28] (C)	Dilithium	136,992	750,998	150,687
3			253,395	1,257,304	255,421
5	[9] (C)	Falcon	35,385,575	1,159,582	105,993
	Our work (Asm)		29,956,252	990,487	62,547
	[28] (C)	Dilithium	356,908	1,439,069	388,523
	Speed-Up ([9] vs Our)		+18.1%	+17.1%	+69.4%

presented a parallel implementation that processes the operations on 8 coefficients simultaneously. In addition, memory accesses that may occur during NTT are minimized by proposing memory optimization paths for each security level. In the point-wise multiplication process, many multiplications were processed at once using a parallel implementation technique that maximized the use of the NEON engine. In the case of point-wise multiplication of NTT that processes 8 coefficients at the same time, since integer-multiplication based on Montgomery reduction is included, the actual performance improvement rate is lower than that of the FFT algorithm.

With these advantages, we achieved performance gains of 177/154% (NTT), 9/8% (Point-Wise multiplication), and 257/234% (InvNTT) in Falcon-512/1024, respectively. In FFT, the lowest level FPC functions are optimized using FP instruction and parallel implementation on NEON engine. In addition, like NTT, we presented memory optimization paths for each security level to minimize the memory access overheads that may occur in FFT. Through these contributions, performance improvements of 254/219% (FFT),

11/10% (Point-Wise multiplication), and 282/267% (InvFFT) were achieved in Falcon-512/1024, respectively. As a result, it shows the superior performance of 250/228% in FFT-based multiplication and 210/118% in NTT-based multiplication of Falcon-512/1024 through the proposed optimization techniques compared to the reference implementation.

C. PERFORMANCE ANALYSIS FOR OVERALL FUNCTIONS OF FALCON

Table 4 shows the performance improvement for the full Falcon scheme. Through optimization implementation, we achieved performance improvements of 15.1/18.1% (KeyGen), 16.5/17.1% (Sign), and 65.4/69.4% (Verify) in Falcon-512/1024, respectively. In Table 3, the performance improvement ratio for FFT/NTT implementations decreases as the security level increases due to the length of the polynomial, but in the case of the overall Falcon scheme, the performance improvement ratio increases as the security level increases. This is because there are various versions of FFT/NTT implementation in Falcon, and the number of

FFT/NTT calls increases as the security level increases. If all optimized FFT/NTT implementations are applied to perform performance measurement, the higher the security level, the greater the benefit of the optimization implementation. Compared to Falcon's competitor, Crystals-Dilithium, despite the optimization method applied, KeyGen process requires a high cost. However, this is not a big consideration since the number of calls to KeyGen in the actual field is very small compared to Sign and Verify process.

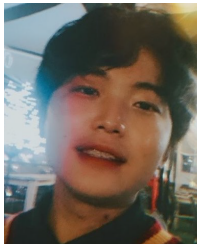
VI. CONCLUSION

In this paper, We proposed a optimal software of NTT/FFT, which is the core operation of Falcon Round 3. From an NTT point of view, it took full advantage of the NEON vector register to process eight coefficients simultaneously, accelerating performance over a reference implementation. From the FFT point of view, we proposed a parallel implementation that processes two coefficients using the Floating Point Instruction Set and NEON vector register supported by the target device. In addition, the Falcon algorithm isn't based on a module structure but NTRU lattice, so the degree of the operation-based polynomial for each security level is different. It is based on the 512 degree polynomial operation (res. 1024 degree polynomial operation) by Security Level 1 (res. Security Level 5). Therefore, a memory-optimized path considering FFT/NTT depth for each security level is required. We propose memory optimization paths for each FFT/NTT for each security level, minimizing the memory access overheads that occurs during transform. Through these contributions, we achieved a performance improvement of 210% by Falcon-512 in NTT and 250% by Falcon-512 in FFT multiplication compared to the FFT/NTT implementation of Falcon Round 3 Submission. As a result, integrating the proposed FFT/NTT optimal implementation technique into Falcon, we achieved 15.1%, 16.5%, and 65.4% performance improvement in KeyGen, Sign, and Verify, respectively, compared to Falcon Round 3 Submission based on Falcon-512. The proposed FFT implementation methodology and the optimization method using the merging and register-holding technique of NTT can be applied to all ARMv8-based devices including ARM-Cortex A series. Our study is meaningful in the results of optimizing FFT/NTT-based multiplication, the core operation of Falcon, for the first time in the ARMv8-A environment. In the future, we plan to optimally implement various PQC algorithms such as lattice-based, code-based, and multi-variable-based in the ARMv8-A environment.

REFERENCES

- [1] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Found. Comput. Sci.*, 1994, pp. 124–134.
- [2] N. Bindel, "Suitability of 3rd round signature candidates for vehicle-to-vehicle communication," in *Proc. 3rd PQC Standardization Conf.*, 2021. [Online]. Available: <https://csrc.nist.gov/Presentations/2021/suitability-of-3rd-round-signature-candidates-for>
- [3] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-quantum authentication in TLS 1.3: A performance study," *Cryptol. ePrint Arch., Tech. Rep.* 2020/071, 2020. [Online]. Available: <https://ia.cr/2020/071>
- [4] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, "pqm4: Testing and benchmarking NIST PQC on ARM cortex-M4," *IACR Cryptol. ePrint Arch.*, vol. 2019, p. 844, Jul. 2019.
- [5] (2021). *PQClean*. [Online]. Available: <https://github.com/PQClean/PQClean>
- [6] (2021). *Liboqs*. [Online]. Available: <https://github.com/open-quantum-safe/liboqs>
- [7] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, 1965.
- [8] W. M. Gentleman and G. Sande, "Fast Fourier transforms: For fun and profit," in *Proc. November Fall Joint Comput. Conf. XX AFIPS (Fall)*, 1966, pp. 563–578.
- [9] T. Prest, P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, (2020). *Falcon Specifications and Supporting Documentation*. [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>
- [10] H. Nejatollahi, N. Dutt, S. Ray, F. Regazzoni, I. Banerjee, and R. Cammarota, "Post-quantum lattice-based cryptography implementations: A survey," *ACM Comput. Surv.*, vol. 51, no. 6, pp. 1–41, Feb. 2019.
- [11] ARM. (2020). *Asimd Vector Instructions*. [Online]. Available: <https://developer.arm.com/documentation/ddi0596/2020-12/SIMD-FP-Instructions>
- [12] ARM. (2020). *A64 Instructions*. [Online]. Available: <https://developer.arm.com/documentation/ddi0596/2020-12/Base-Instructions>
- [13] T. Oder, J. Speith, K. Höltingen, and T. Güneysu, "Towards practical microcontroller implementation of the signature scheme falcon," in *Post-Quantum Cryptography*, J. Ding and R. Steinwandt, Eds. Cham, Switzerland: Springer, 2019, pp. 65–80.
- [14] S. Streith and F. D. Santis, "Post-quantum key exchange on ARMv8-A: A new hope for NEON made simple," *IEEE Trans. Comput.*, vol. 67, no. 11, pp. 1651–1662, Nov. 2018.
- [15] P. Sanal, E. Karagoz, H. Seo, R. Azarderakhsh, and M. Mozaffari-Kermani, "Kyber on ARM64: Compact implementations of Kyber on 64-bit ARM Cortex-A processors," *Cryptol. ePrint Arch., Tech. Rep.* 2021/561, 2021. [Online]. Available: <https://ia.cr/2021/561>
- [16] H. Becker, V. Hwang, M. J. Kannwischer, B.-Y. Yang, and S.-Y. Yang, "Neon NTT: Faster Dilithium, Kyber, and Saber on Cortex-A72 and Apple M1," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2022, no. 1, pp. 221–244, 2021, doi: [10.46586/tches.v2022.i1.221-244](https://doi.org/10.46586/tches.v2022.i1.221-244).
- [17] D. T. Nguyen and K. Gaj, "Optimized software implementations of CRYSTALS-Kyber, NTRU, and Saber using NEON-based special instructions of ARMv8," in *Proc. NIST 3rd PQC Standardization Conf. (NIST PQC)*, Jun. 2021, pp. 1–24.
- [18] N. Gupta, A. Jati, A. K. Chauhan, and A. Chattopadhyay, "PQC acceleration using GPUs: FrodoKEM, NewHope, and kyber," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 3, pp. 575–586, Mar. 2021.
- [19] S. An and S. C. Seo, "Efficient parallel implementations of LWE-based post-quantum cryptosystems on graphics processing units," *Mathematics*, vol. 8, no. 10, p. 1781, Oct. 2020.
- [20] P. Duong-Ngoc, T. N. Tan, and H. Lee, "Efficient NewHope cryptography based facial security system on a GPU," *IEEE Access*, vol. 8, pp. 108158–108168, 2020.
- [21] S. C. Seo, "SIKE on GPU: Accelerating supersingular isogeny-based key encapsulation mechanism on graphic processing units," *IEEE Access*, vol. 9, pp. 116731–116744, 2021.
- [22] W.-K. Lee, S. Akleyek, W.-S. Yap, and B.-M. Goi, "Accelerating number theoretic transform in GPU platform for qTESLA scheme," in *Information Security Practice and Experience*, S.-H. Heng and J. Lopez, Eds. Cham, Switzerland: Springer, 2019, pp. 41–55.
- [23] H. Cheng, J. Großschädl, P. B. Rønne, and P. Y. A. Ryan, "AVR-NTRU: Lightweight NTRU-based post-quantum cryptography for 8-bit AVR microcontrollers," in *Proc. Design, Automat. Test Eur. Conf. Exhib. (DATE)*, 2021, pp. 1272–1277, doi: [10.23919/DATES1398.2021.9474033](https://doi.org/10.23919/DATES1398.2021.9474033).
- [24] N. Gura, A. Patel, A. Wander, H. Eberle, and S. C. Shantz, "Comparing elliptic curve cryptography and RSA on 8-bit cpus," in *Cryptographic Hardware and Embedded Systems CHES*, M. Joye and J.-J. Quisquater, Eds. Berlin, Germany: Springer, 2004, pp. 119–132.
- [25] C.-M.-M. Chung, V. Hwang, M. J. Kannwischer, G. Seiler, C.-J. Shih, and B.-Y. Yang, "NTT multiplication for NTT-unfriendly rings: New speed records for saber and NTRU on cortex-M4 and AVX2," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2021, pp. 159–188, Feb. 2021.

- [26] J. Zhang, J. Huang, Z. Liu, and S. S. Roy, "Time-memory trade-offs for Saber+ on memory-constrained RISC-V platform," *IEEE Trans. Comput.*, early access, Jan. 14, 2022, doi: [10.1109/TC.2022.3143441](https://doi.org/10.1109/TC.2022.3143441).
- [27] D. O. C. Greconici, M. J. Kannwischer, and D. Sprenkels, "Compact dilithium implementations on cortex-M3 and cortex-M4," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2021, pp. 1–24, Dec. 2020.
- [28] S. Bai, L. Ducas, E. Kiltz, T. R. Lcpoint, P. Schwabe, G. Seiler, and D. Stehle. (2020). *Crystals-Dilithium Algorithm Specifications and Supporting Documentation*. [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>
- [29] *NVIDIA AGX Xavier Specification*. Accessed: Nov. 11, 2021. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-agx-xavier-developer-kit>



YOUNGBEOM KIM (Student Member, IEEE) received the B.S. degree from the Department of Information Security, Cryptology, and Mathematics, Kookmin University, where he is currently pursuing the master's degree in financial information security. His research interests include optimization of cryptographic algorithms, its efficient implementations on various IoT devices, and cryptographic module validation program.



JINGYO SONG (Student Member, IEEE) received the bachelor's degree from the Department of Information Security, Cryptology, and Mathematics, Kookmin University, Seoul, South Korea, and the master's degree from the Department of Financial Information Security, Kookmin University. His research interests include optimization of cryptographic algorithms, post-quantum cryptography, and designing efficient side-channel attacks countermeasures in ARM-cortex A series.



SEOG CHUNG SEO (Member, IEEE) received the B.S. degree in information and computer engineering from Ajou University, Suwon, South Korea, in 2005, the M.S. degree in information and communications from the Gwangju Institute of Science and Technology (GIST), Gwangju, South Korea, in 2007, and the Ph.D. degree from Korea University, Seoul, South Korea, in 2011. He worked as a Research Staff Member at Samsung Advanced Institute of Technology (SAIT) and Samsung DMC Research and Development Center, from September 2011 to April 2014. He was a Senior Research Member of the Affiliated Institute of ETRI, South Korea, from 2014 to 2018. Currently, he works as an Associate Professor at Kookmin University, Seoul. His research interests include public-key cryptography, its efficient implementations on various IT devices, cryptographic module validation program, network security, and data authentication algorithms.

...